



Pets Expression Classification

Computer Vision Assignment 4

Faculty of Engineering

Alexandria University

2025–2026 Academic Year

Prepared By

Student Name	Student ID
Mohammed Adel	9158
Nader Ahmed	9111
Mahmoud Walled	8958

Contents

1	Introduction	3
2	Dataset Description	3
2.1	Dataset Information	3
2.2	Sample Images	4
3	Data Preprocessing and Augmentation	4
3.1	Preprocessing	4
3.2	Data Augmentation	5
4	Implemented Models	6
4.1	VGG16-BN	6
4.2	ResNet18	7
4.3	MobileNetV1	8
4.4	InceptionV3	8
4.5	DenseNet121	9
5	Training Configuration	10
5.1	Optimization Strategy	10
5.2	Loss Function	10
5.3	Learning Rate Scheduling	10
5.4	Training Procedure	11
5.5	Evaluation Metrics	11
5.6	General Hyperparameters	12
6	Results	12
6.1	Performance Comparison	12
6.2	Confusion Matrices and Training Curves	13
7	Transfer Learning	17
7.1	Transfer Learning Results	18
7.2	Performance and Convergence Analysis	18
7.3	Detailed Class-Wise Evaluation	19
7.3.1	Confusion Matrix Key Insights	19

1 Introduction

In this assignment, different deep learning models were implemented and evaluated for pets facial expression classification. The objective is to classify pet expressions using convolutional neural networks and compare the performance of multiple well-known architectures.

The assignment also investigates the impact of transfer learning using pretrained ImageNet weights.

2 Dataset Description

The dataset contains face images for different pets such as dogs, cats, rabbits, horses, birds, and others. The images represent different facial expressions and emotional states.

2.1 Dataset Information

- Number of images: 1000 images
- Number of classes: 4 classes (Angry, Other, Sad, happy)
- Image size: 224×224 pixels (RGB)
- Train/Validation/Test split: 70% Training, 15% Validation, 15% Testing

2.2 Sample Images



Figure 1: Sample images from the dataset

3 Data Preprocessing and Augmentation

Data preprocessing and augmentation were implemented using the PyTorch `torchvision.transforms` pipeline. These operations were applied dynamically during training to improve model generalization, reduce overfitting, and increase robustness against spatial and illumination variations.

3.1 Preprocessing

The following preprocessing operations were applied:

- **Image Resizing:** Images were first resized to 256×256 pixels before cropping operations. This provides additional spatial flexibility during augmentation.

- **Random Cropping:** During training, random crops of size 224×224 were extracted from resized images to improve spatial invariance and reduce overfitting.
- **Center Cropping:** During evaluation, a deterministic center crop of 224×224 was applied to ensure stable and reproducible validation/testing results.
- **Tensor Conversion:** Images were converted into PyTorch tensors using `ToTensor()`.
- **Normalization:** Images were normalized using the standard ImageNet normalization statistics:

$$\mu = [0.485, 0.456, 0.406]$$

$$\sigma = [0.229, 0.224, 0.225]$$

This normalization is commonly used for models trained or initialized using ImageNet weights.

3.2 Data Augmentation

Several augmentation techniques were applied during training:

- **Random Horizontal Flip** ($p = 0.5$): Improves robustness against horizontal orientation changes.
- **Random Rotation** ($\pm 10^\circ$): Introduces rotational variance while preserving semantic facial structures.
- **Color Jitter:** Random modifications to:
 - Brightness ($\pm 25\%$)
 - Contrast ($\pm 25\%$)
 - Saturation ($\pm 10\%$)
 - Hue ($\pm 5\%$)

These augmentations simulate illumination and color distribution variations commonly encountered in real-world image acquisition.

4 Implemented Models

The following convolutional neural network architectures were implemented and trained using PyTorch.

4.1 VGG16-BN

A custom VGG16-style architecture with Batch Normalization was implemented from scratch. The network is composed of five sequential convolutional blocks following the classical VGG configuration:

$$[2, 2, 3, 3, 3]$$

convolutional layers per stage.

Each block utilizes:

- 3×3 convolutions
- Batch Normalization
- ReLU activation
- 2×2 MaxPooling

The spatial dimensions are progressively reduced from:

$$224 \times 224 \rightarrow 112 \times 112 \rightarrow 56 \times 56 \rightarrow 28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7$$

The classifier head consists of:

- Fully connected layer:

$$512 \times 7 \times 7 \rightarrow 4096$$

- ReLU activation + Dropout
- Second fully connected layer:

$$4096 \rightarrow 4096$$

- Final classification layer:

$$4096 \rightarrow \text{num_classes}$$

Batch Normalization was integrated to improve optimization stability and accelerate convergence compared to the original VGG16 architecture.

4.2 ResNet18

A ResNet18 architecture was implemented from scratch using residual learning principles to address optimization degradation in deep convolutional networks.

The architecture is built using residual blocks composed of two sequential 3×3 convolutional layers combined with identity shortcut connections. Instead of directly learning a full transformation, each block learns a residual mapping that is added to the original input through skip connections.

Each residual block contains:

- 3×3 convolution
- Batch Normalization
- ReLU activation
- Identity shortcut connection

When feature dimensions or spatial resolutions change, projection shortcuts using 1×1 convolutions are applied to align tensor dimensions before residual addition.

The overall architecture follows the standard ResNet18 stage configuration:

$$[2, 2, 2, 2]$$

residual blocks across four feature extraction stages.

The network progressively increases channel depth:

$$64 \rightarrow 128 \rightarrow 256 \rightarrow 512$$

while spatial dimensions are reduced through strided convolutions and pooling operations.

After the final residual stage, Adaptive Average Pooling reduces the feature maps into a compact representation before the final fully connected classification layer:

$$512 \rightarrow \text{num_classes}$$

Residual learning improves gradient propagation through deep networks and helps stabilize optimization during training.

4.3 MobileNetV1

A MobileNetV1-inspired architecture was implemented from scratch using depthwise separable convolutions to significantly reduce computational complexity and parameter count.

Instead of applying standard convolutions directly, each convolutional operation is decomposed into:

- **Depthwise Convolution:** Independent spatial filtering applied separately to each input channel using grouped convolutions.
- **Pointwise Convolution:** A 1×1 convolution used to linearly combine depthwise feature maps across channels.

This decomposition greatly lowers computational cost compared to standard convolutions while preserving representational capability.

The architecture progressively expands feature channels through stages:

$$32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024$$

while spatial resolution is reduced using strided depthwise convolutions.

Global Adaptive Average Pooling is applied before the final linear classification layer:

$$1024 \rightarrow \text{num_classes}$$

The lightweight design of MobileNet makes it computationally efficient while maintaining competitive classification performance.

4.4 InceptionV3

The **InceptionV3** architecture optimizes deep convolutional networks by using **factorized, multi-scale feature extraction**. Instead of using a single generic block layout, it deploys specialized Inception modules tailored to specific spatial dimensions to drastically reduce parameters while maximizing information flow.

Key Structural Blocks:

- **Convolutional Stem:** A deep sequence of initial layers that aggressively downsamples raw inputs from $299 \times 299 \times 3$ to a compact $35 \times 35 \times 192$ feature map before it reaches the main blocks.
- **InceptionA (35×35 Grid):** Replaces large 5×5 convolutions with two stacked, efficient 3×3 layers.
- **InceptionC (17×17 Grid):** Introduces **asymmetric convolutions** (factorizing 7×7 layers into parallel 1×7 and 7×1 sequences) to cut parameter weight by nearly 33%.
- **InceptionE (8×8 Grid):** Uses parallelized asymmetric branches that split wide channels laterally to synthesize high-level semantic features.
- **Grid Reduction Blocks (B & D):** Specialized modules that run pooling and strided convolutions concurrently to prevent data loss or architectural bottlenecks during downsampling.

Network Flow Summary:

- **Spatial Downsampling:** $299 \times 299 \rightarrow 149 \times 149 \rightarrow 147 \times 147 \rightarrow 73 \times 73 \rightarrow 71 \times 71 \rightarrow 35 \times 35 \rightarrow 17 \times 17 \rightarrow 8 \times 8 \rightarrow 1 \times 1$
- **Channel Expansion:** $3 \rightarrow 32 \rightarrow 64 \rightarrow 80 \rightarrow 192 \rightarrow 288 \rightarrow 768 \rightarrow 1280 \rightarrow 2048$
- **Classifier Head:** Employs Adaptive Average Pooling, 50% Dropout, and a final Linear layer ($2048 \rightarrow \text{num_classes}$).

4.5 DenseNet121

The **DenseNet121** (Dense Convolutional Network) architecture fundamentally alters the way layers are connected compared to traditional CNNs. It connects each layer to every other layer in a feed-forward fashion. To preserve the feed-forward nature, each layer obtains additional inputs from all preceding layers and passes on its own feature-maps to all subsequent layers. This dense connectivity pattern significantly alleviates the vanishing-gradient problem, strengthens feature propagation, encourages feature reuse, and substantially reduces the number of overall parameters compared to ResNet architectures.

5 Training Configuration

All models were trained using PyTorch on GPU acceleration when available. A unified training pipeline was adopted to ensure fair comparison across architectures.

5.1 Optimization Strategy

The **Adam** optimizer was utilized for all experiments due to its adaptive learning capability and stable convergence behavior in deep neural network optimization.

The optimizer configuration included:

- Initial Learning Rate:

$$1 \times 10^{-3}$$

- Weight Decay (L2 Regularization):

$$1 \times 10^{-5}$$

Weight decay was applied to reduce overfitting and improve generalization performance.

5.2 Loss Function

The classification task was optimized using weighted categorical cross entropy implemented through:

$$\text{CrossEntropyLoss}$$

To address dataset imbalance, inverse-frequency class weights were computed from the training dataset and integrated directly into the loss function.

Additionally, **Label Smoothing** with smoothing factor:

$$\alpha = 0.1$$

was applied to reduce model overconfidence and improve generalization.

5.3 Learning Rate Scheduling

A **Cosine Annealing Warm Restarts** scheduling strategy was employed to dynamically adjust the learning rate during training.

The scheduler periodically reduces and restarts the learning rate according to a cosine decay profile, helping the optimizer escape shallow local minima and improve convergence stability.

The scheduler configuration was:

- Initial restart period:

$$T_0 = 10$$

- Restart multiplier:

$$T_{mult} = 1$$

- Minimum learning rate:

$$\eta_{min} = 10^{-6}$$

5.4 Training Procedure

For each epoch, the workflow consisted of:

1. Training phase on the training dataset
2. Validation phase on the validation dataset
3. Learning rate scheduler update
4. Best model checkpointing based on validation loss

The model state achieving the lowest validation loss was automatically saved and restored before final testing evaluation.

5.5 Evaluation Metrics

The following evaluation metrics were computed:

- Classification Accuracy
- Precision (Macro Average)
- Recall (Macro Average)
- F1-Score (Macro Average)
- Confusion Matrix

Training and validation loss/accuracy curves were also visualized to monitor optimization behavior and detect overfitting or underfitting trends.

5.6 General Hyperparameters

Table 1: General Training Hyperparameters

Parameter	Value
Optimizer	Adam
Initial Learning Rate	1×10^{-3}
Minimum Learning Rate	1×10^{-6}
Weight Decay	1×10^{-5}
Loss Function	Weighted Cross Entropy
Label Smoothing	0.1
Scheduler	CosineAnnealingWarmRestarts
Batch Size	32
Epochs	20
Checkpoint Criterion	Lowest Validation Loss
Evaluation Metrics	Accuracy, Precision, Recall, F1-Score

6 Results

6.1 Performance Comparison

Table 2: Models Performance (Built From Scratch)

Model	Accuracy	Precision	Recall	F1-Score
VGG16	0.2368	0.12	0.24	0.15
ResNet18	0.3947	0.39	0.41	0.39
MobileNet	0.2895	0.31	0.28	0.28
DenseNet121	0.37	0.36	0.35	0.35
InceptionV3	0.3684	0.38	0.36	0.37

6.2 Confusion Matrices and Training Curves

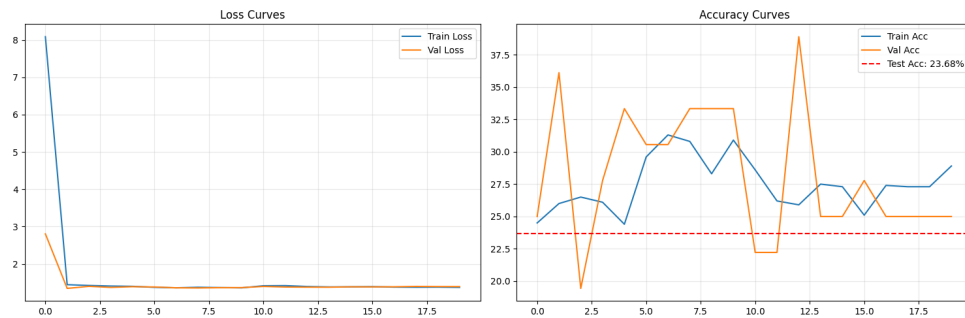


Figure 2: VGG16 Learning Curves

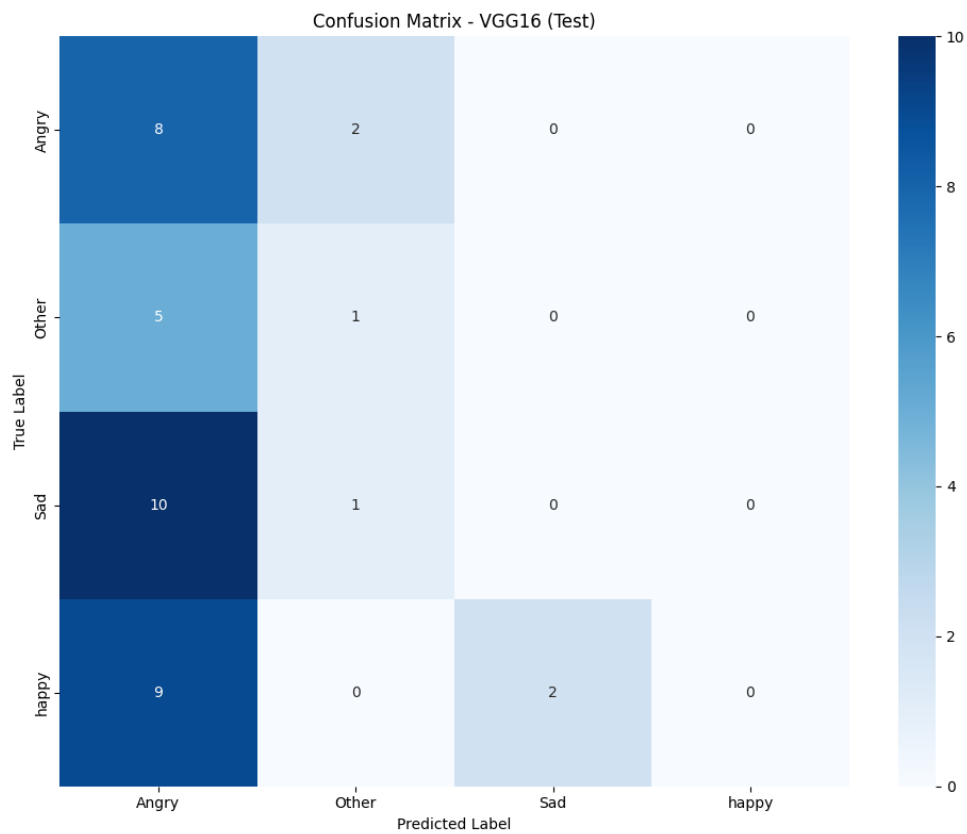


Figure 3: VGG16 Confusion Matrix

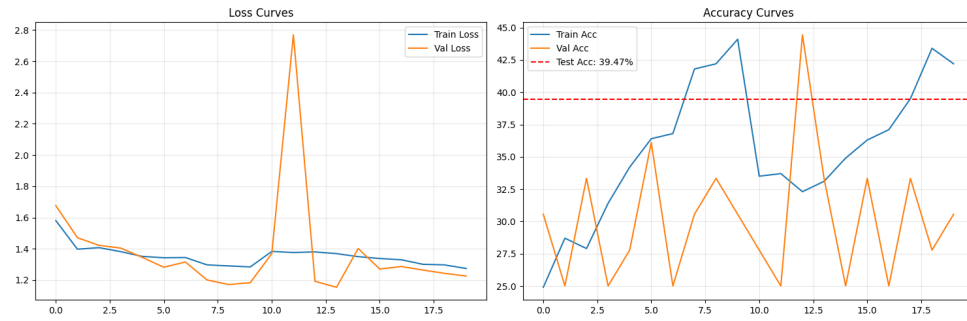


Figure 4: ResNet18 Learning Curves

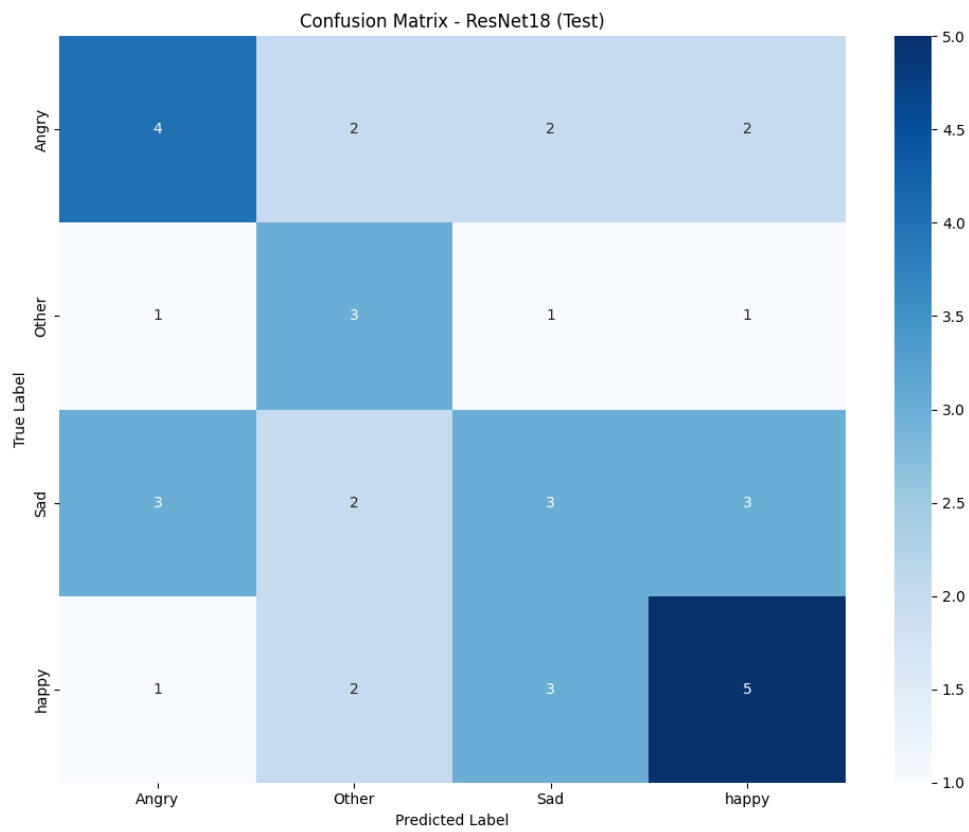


Figure 5: ResNet18 Confusion Matrix

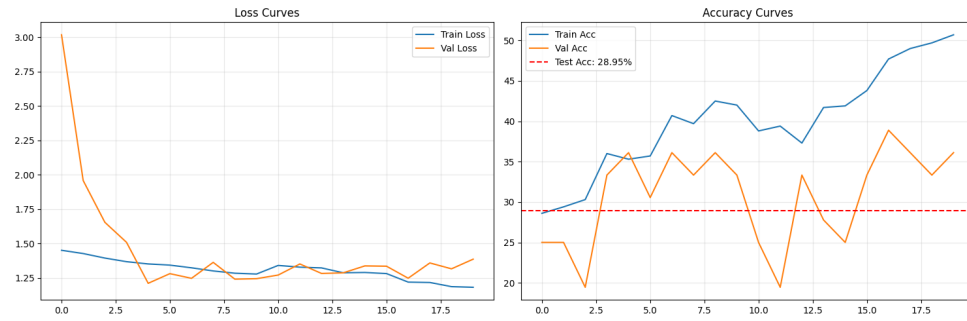


Figure 6: MobileNet Learning Curves

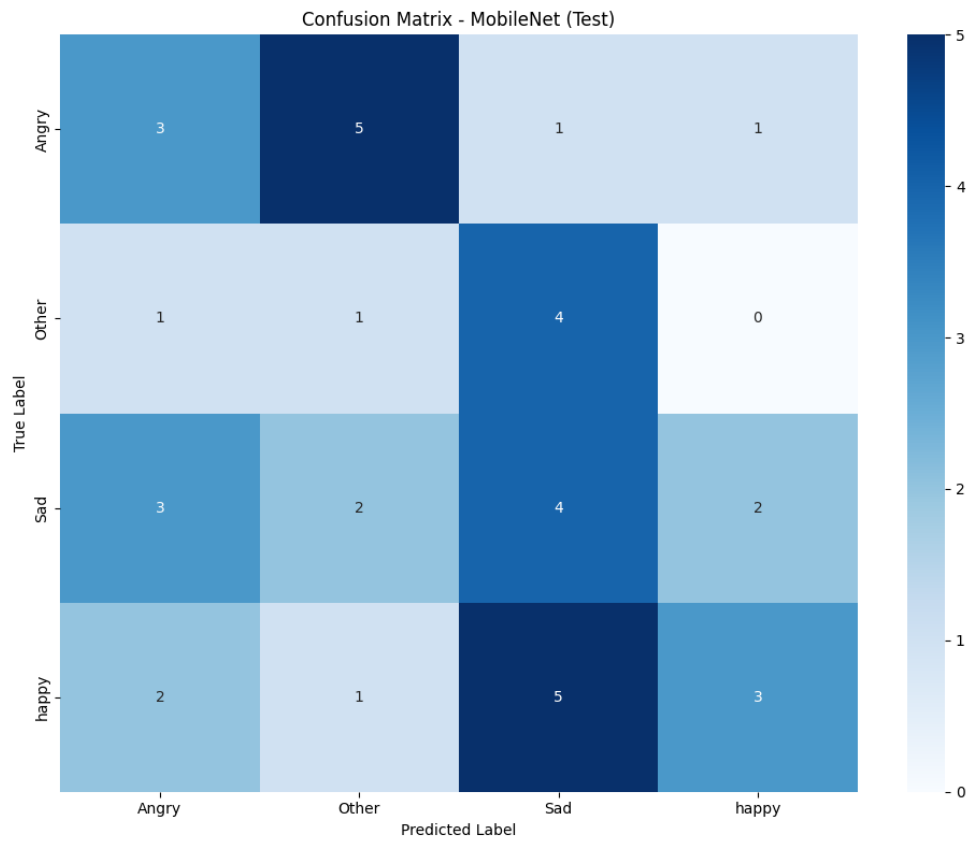


Figure 7: MobileNet Confusion Matrix

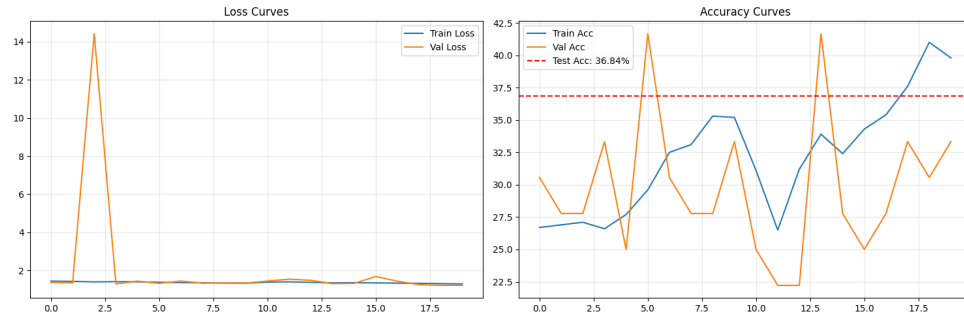


Figure 8: InceptionV3 Learning Curves

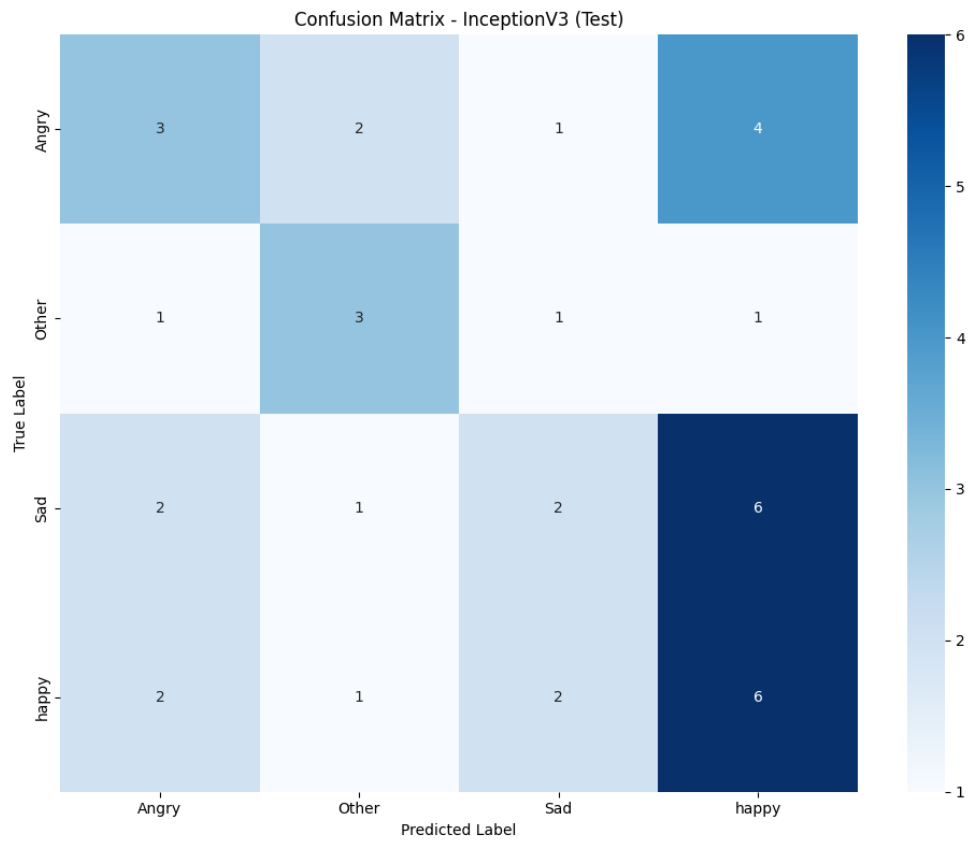


Figure 9: InceptionV3 Confusion Matrix

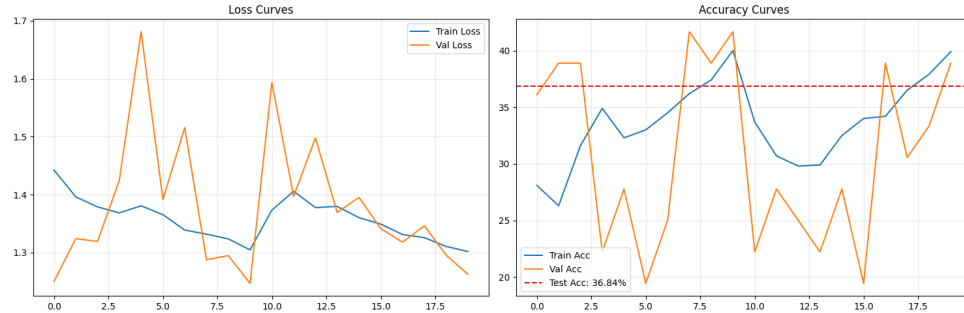


Figure 10: DenseNet121 Learning Curves

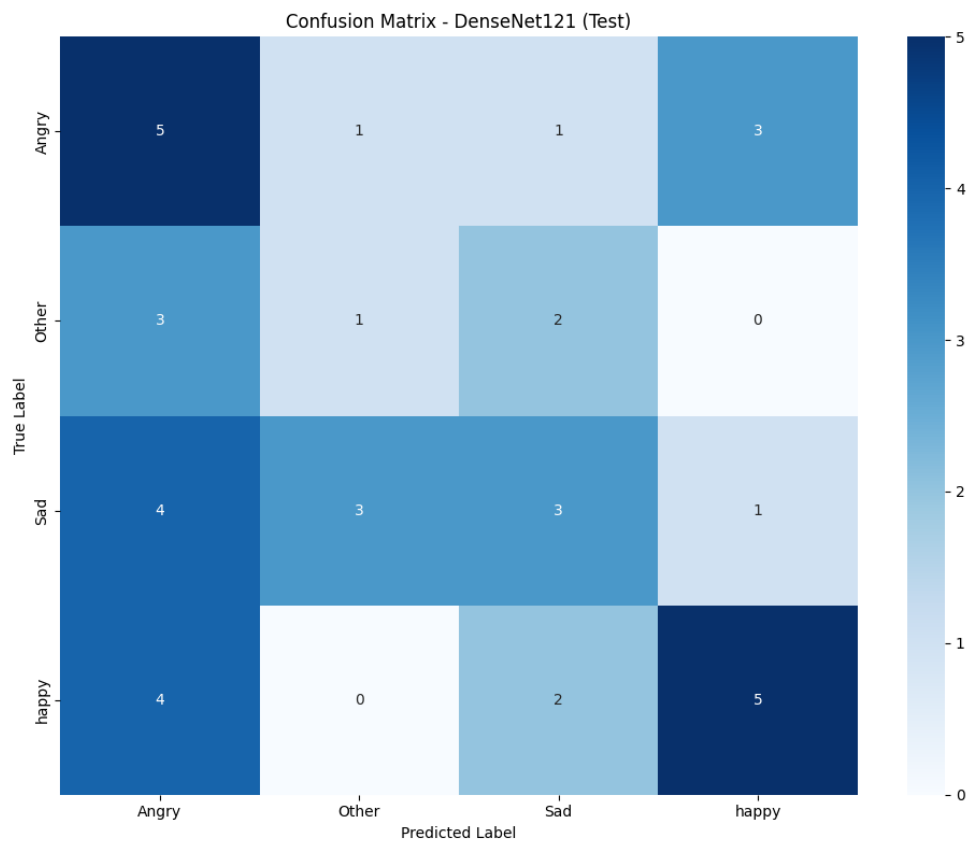


Figure 11: DenseNet121 Confusion Matrix

7 Transfer Learning

Transfer learning was investigated using ImageNet pretrained weights. Instead of initializing model parameters randomly, pretrained feature extractors learned from the ImageNet dataset were reused and adapted to the pets facial expression classification task.

The final classification layer was replaced to match the number of target classes in the dataset, and the network was subsequently fine-tuned on the custom dataset.

Transfer learning improves convergence speed and generalization capability, especially when training data is limited, since pretrained networks already capture rich low-level and mid-level visual representations such as edges, textures, shapes, and semantic object structures.

7.1 Transfer Learning Results

To evaluate the impact of transfer learning, the performance of the best model trained from scratch (*ResNet18*) was compared against the pre-trained *DenseNet121* fine-tuned using discriminative learning rates and spatial layout adjustments.

The evaluation metrics demonstrate that leveraging pre-trained ImageNet weights provides a substantial performance boost, breaking past the strict performance limitations observed when training complex architectures from scratch on a limited data regime.

Table 3: Performance Before and After Transfer Learning

Metric (Macro Average)	Before (Best Scratch: ResNet18)	After (DenseNet121 Transfer)
Accuracy	0.3947	0.5000
Precision	0.3900	0.4700
Recall	0.4100	0.4600
F1-Score	0.3900	0.4600

7.2 Performance and Convergence Analysis

The training behavior of the pre-trained DenseNet121 model highlights distinct optimization trends over the course of 20 epochs:

- **Loss Convergence:** The training loss demonstrates a continuous, highly stable decay curve, decreasing smoothly from an initial value of approximately 1.40 down to 0.88. Conversely, the validation loss reaches its global minimum floor near Epoch 13 (1.1645) before displaying a minor upward inflection, indicating the boundary where additional parameters start over-fitting the localized image styles.
- **Generalization Leap:** While training accuracy approaches an upper bounds of over 70%, validation accuracy successfully hits a stable plateau of 47.22%, en-

ensuring that the checkpointing logic saves the configuration at its optimal point of generalizability.

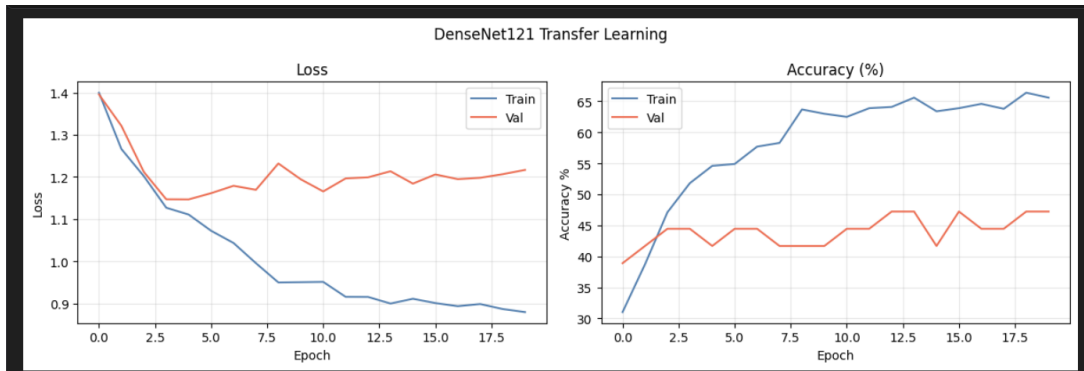


Figure 12: DenseNet121 Transfer Learning Loss and Accuracy Curves

7.3 Detailed Class-Wise Evaluation

The test set evaluation yields a comprehensive look at how well the fine-tuned feature extractor isolates pet facial markers across classes:

Table 4: DenseNet121 Fine-Tuning Classification Report

Class Name	Precision	Recall	F1-Score	Support
Angry	0.50	0.60	0.55	10
Other	0.33	0.17	0.22	6
Sad	0.54	0.64	0.58	11
Happy	0.50	0.45	0.48	11
Accuracy			0.50	38
Macro Avg	0.47	0.46	0.46	38
Weighted Avg	0.48	0.50	0.49	38

7.3.1 Confusion Matrix Key Insights

Looking deeply at the resulting test confusion matrix, clear behavioral correlations emerge:

- **Strongest Classes:** The model proves most adept at isolating *Sad* expressions, yielding a peak F1-score of 0.58 and accurately predicting 7 out of 11 true occurrences. This is followed closely by the *Angry* class with a recall score of 0.60.
- **Common Misclassifications:** A strong systemic confusion occurs between the *Happy* and *Sad* classes, where 4 true happy images were mistakenly categorized

as sad. This points to shared geometric mouth/eye traits across varying domestic animal phenotypes.

- **The Open Class Weakness:** The *Other* class yielded poor performance metrics (Recall of 0.17), with images widely scattered across all alternative categories. This is expected, as an ambiguous category contains vastly conflicting background contexts and edge boundaries, making structured cluster grouping highly challenging.

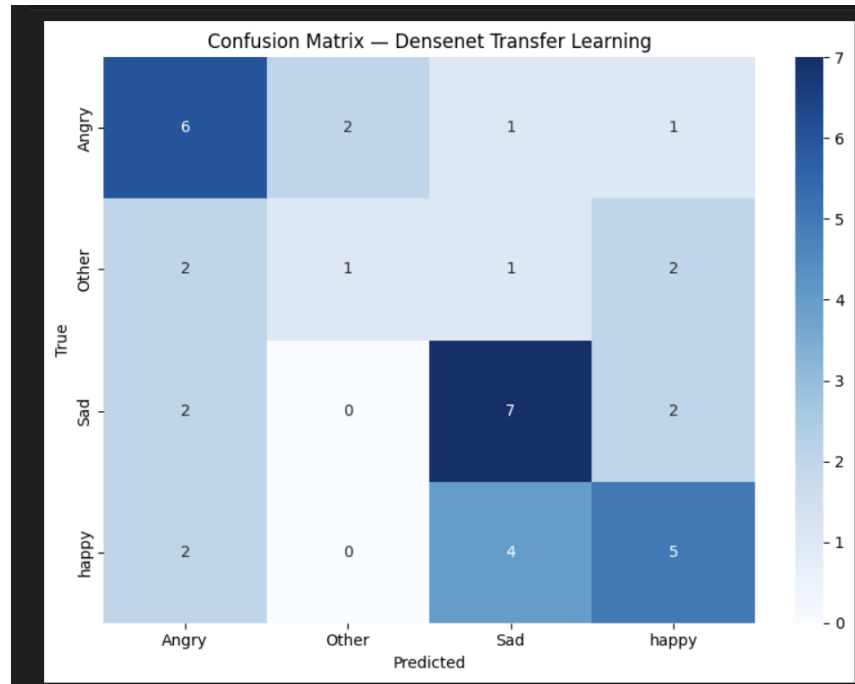


Figure 13: DenseNet121 Transfer Learning Confusion Matrix